

멀티 스레드 환경 안드로이드 어플리케이션 분석을 위한 Stacktrace 추적 기법 연구*

윤성배* 박제만*

*경희대학교 (대학원생) *경희대학교 (교수)

A Stacktrace Tracking Technique for Multi-Threaded Android Application Analysis

SungBae Yoon* Jeman Park*

*Kyung Hee University (Graduate student) *Kyung Hee University (Professor)

요약

본 연구는 안드로이드 어플리케이션의 동적 분석 과정에서 멀티 스레드 환경으로 인해 발생하는 스택트레이스 단절 문제를 해결하기 위한 기법을 제안한다. Frida 기반 후킹을 활용하여, 스레드 생성 시점의 외부 스택 정보와 실행 시점의 내부 스택 정보를 병합함으로써 전체 호출 경로를 복원하였다. 이를 통해 API 호출의 실제 발생 지점과 실행 흐름을 정확히 파악할 수 있으며, 동적 분석 기법에서 발생 가능한 비동기 호출 추적 한계를 극복하였다. 제안된 방법은 악성 행위 탐지뿐만 아니라 일반 어플리케이션의 런타임 행위 분석 및 성능 진단에도 활용 가능하다.

I. 서론

안드로이드 어플리케이션의 동적 분석은 어플리케이션이 실제 실행되는 환경에서 발생하는 API 호출, 이벤트 처리, 네트워크 통신 등의 행위를 관찰함으로써 시스템의 동작 특성을 이해하고, 잠재적 오류나 비정상적 행위를 탐지하기 위한 핵심 기술이다.

기존에는 주로 악성 어플리케이션 분석 분야에서 이러한 기법이 활용되어 왔으나, 최근에는 일반 어플리케이션의 성능 최적화, 보안 검증, 런타임 행위 추적 등 다양한 분야에서도 동적 분석의 수요가 증가하고 있다.

1.1 안드로이드 어플리케이션 분석 방법

안드로이드 어플리케이션 분석 방법은 크게 정적 분석과 동적 분석으로 구분된다.

정적 분석은 어플리케이션을 실행하지 않고

APK 내부의 코드와 메타데이터를 분석하여 구조를 파악한다. 정적 분석은 빠르고 높은 코드 커버리지에 대한 분석이 가능하지만 코드 난독화나 동적 로딩과 같은 회피 기법에 대해 한계를 노출한다. 또한 실행 파일만 존재하는 경우, 이를 코드로 되돌리기 위한 역공학 과정이 요구된다.

반면 동적 분석은 프로그램을 실행시켜 런타임에서 나타나는 행동을 직접 확인할 수 있다. 난독화된 코드, 동적 로딩, 비정상적 런타임 동작의 원인 파악에 유리하기 때문에, 어플리케이션 분석에서 넓게 활용된다. 특히, 정적 분석과 동적 분석을 결합하여 상호 보완적 방식으로 사용하는 경우도 많다.

1.2 동적 분석의 한계

동적 분석 시에는 특정 API가 어떤 맥락에

* "본 연구는 과학기술정보통신부 및 정보통신기획평가원의 융합보안핵심인재양성사업의 연구 결과로 수행되었음" (IITP-2025-RS-2023-00266615*)

서 사용되는지를 확인하는 것이 중요하다. 이때, 분석가는 현재 런타임의 스택트레이스(stacktrace)를 확인할 수 있다. 스택트레이스에는 API가 호출되기까지 프로그램의 실행 흐름이 나타나며, 어플리케이션의 행동 파악에 대한 중요한 단서를 제공한다.

그러나 현대 안드로이드 어플리케이션은 비동기 처리, 멀티 스레딩, 동적 로딩 등 복잡한 구조를 통해 실행된다. Java Virtual Machine (JVM) 기반의 스레드 모델을 따르는 안드로이드에서 각 스레드는 독립된 호출 스택(call stack)을 가진다. 이로 인해, 멀티 스레드 환경에서 특정 API 호출의 스택트레이스를 확인하는 경우, 해당 스레드가 실행되는 시점에서 분기된 스택트레이스를 보게 된다. 이와 같은 어플리케이션 전체의 행동 맥락을 이해할 수 없는 한계를 갖게 된다.

이에 본 연구에서는 동적 분석 환경에서 멀티 스레드 환경에서도 완전한 API 호출 경로를 복원할 수 있는 추적 기법을 제안한다.

1.3 관련 연구

ECHO^[1]는 비동기 흐름을 추적해야 하는 기능이 필요하여 Xposed를 사용한다. 하지만 Xposed는 Java 레벨 후킹에 한정되어 네이티브 코드 추적이 어렵고, 복잡한 비동기 및 멀티스레드 환경에서 호출 흐름 연결에 한계가 있다. 본 연구에서는 Frida를 사용하여 네이티브와 Java 스택을 통합적으로 추적하고, 동적 스크립트 삽입으로 복잡한 비동기 흐름에서도 더 정밀하고 완전한 추적을 가능하게 하였다.

II. 본론

2.1 Frida

Frida는 동적 바이너리 인스트루멘테이션(Dynamic Binary Instrumentation)기반의 분석 프레임워크로, 런타임 중 함수 호출을 가로채거나 수정할 수 있다. 이는 어플리케이션을 리패키징하거나 재빌드하지 않고도 런타임 후킹을 가능하게 하며, Java 계층뿐 아니라 Native 계층(JNI, C/C++)까지 조작할 수 있다.

안드로이드 환경에서는 Java.use()를 통해 Java 클래스 인스턴스를 동적으로 참조하고, 해당 메서드 호출을 후킹하여 인자와 반환값, 그리고 호출 스택 정보를 실시간으로 수집할 수 있다.

2.2 스택트레이스 추출과 분석

Frida에서는 후킹된 함수 호출 시점에서 Exception().getStackTrace()를 활용하여 현재 호출 경로를 배열 형태로 수집할 수 있다. 이 정보를 활용하면 함수가 어떤 클래스/메서드로부터 호출되었는지 파악할 수 있다.

2.2.1 단일 스레드 환경에서의 스택트레이스

```
at android.location.LocationManager.getLastKnownLocation(
at com.example.testapp.MainActivity.getLocationUsingLocat
at com.example.testapp.MainActivity.handleResultAction(Ma
at com.example.testapp.MainActivity.calculateResult(MainA
at com.example.testapp.MainActivity.onCreate$lambda$3(Mai
at com.example.testapp.MainActivity.$r8$lambda$1Gp8cWalbT
at com.example.testapp.MainActivity$$ExternalSyntheticLam
at android.view.View.performClick(View.java:7542)
at com.google.android.material.button.MaterialButton.perf
at android.view.View.performClickInternal(View.java:7519)
at android.view.View.-$$Nest$performClickInternal(Unknown
at android.view.View$PerformClick.run(View.java:29476)
at android.os.Handler.handleCallback(Handler.java:942)
at android.os.Handler.dispatchMessage(Handler.java:99)
at android.os.Looper.loopOnce(Looper.java:201)
at android.os.Looper.loop(Looper.java:288)
at android.app.ActivityThread.main(ActivityThread.java:79
at java.lang.reflect.Method.invoke(Native Method)
at com.android.internal.os.RuntimeInit$MethodAndArgsCalle
at com.android.internal.os.ZygoteInit.main(ZygoteInit.jav
```

[그림 1] 단일 스레드 환경에서의 스택트레이스

[그림 1]은 단일 스레드 환경을 사용하는 안드로이드 어플리케이션(com.example.testapp)에서 추출된 스택트레이스를 보여준다. getLastKnownLocation()이라는 API가 호출된 시점의 스택트레이스를 출력한 결과, 호출 위치가 com.example.test.app.MainActivity로 나타난다. 출력된 호출 스택의 최하단에는 항상 ZygoteInit.main() 메서드가 위치하는데, 이는 모든 안드로이드 어플리케이션 프로세스가 시스템의 Zygote 프로세스에서 파생되어 실행되기 때문이다. ZygoteInit.main()은 어플리케이션 프로세스의 루트 엔트리 포인트로서, 모든 스택트레이스의 기저에 나타난다. 이러한 결과는 단일 스레드 환경에서 함수 호출 경로가 어플리케이션 전체에 걸쳐 정상적으로 연결되어 있음

을 보여준다.

2.2.2 멀티 스레드 환경에서의 스택트레이스

```
at android.telephony.TelephonyManager.getNetworkOperator(  
at com.example.testapp.MainActivity.getNetworkOperatorInT  
at com.example.testapp.MainActivity.handleResultAction$la  
at com.example.testapp.MainActivity.$r8$lambda$6Rj9kcFWPu  
at com.example.testapp.MainActivity$$ExternalSyntheticLam  
at java.lang.Thread.run(Thread.java:1012)
```

[그림 2] 멀티 스레드 환경에서의 스택트레이스

[그림 2]는 동일한 어플리케이션에서 멀티 스레드 환경에서의 스택트레이스를 보여준다. 비동기로 실행되는 별도의 스레드 내부에서 타겟 API인 `getNetworkOperator()`가 호출되었을 때의 호출 스택이다. 이때의 스택트레이스는 단일 스레드 환경과 달리 `Thread.run()` 메시드가 가장 하단에 배치된다. 멀티 스레드 환경에서는 `Thread.run()`, `Handler.dispatchMessage()` 등 비동기 흐름이 추가로 생성되며, 이 시점에서 기존 호출 스택이 단절된다. 즉, 전체 어플리케이션 관점에서의 호출 스택 (`ZygoteInit.main()`로 시작)이 아닌 현재 스레드의 생성 시점으로부터의 호출 스택만 나타나는 것으로 볼 수 있다.

이와 같은 멀티 스레드 환경에서의 흐름 단절로 인해, 특정 API 호출의 실제 발생 지점을 스택트레이스만으로 역추적하는 것은 불가능해진다. 이는 악성 코드 등을 분석하는 과정에서 전체적인 어플리케이션의 실행 흐름을 이해하기 어렵게 만들고, 정확한 동적 분석의 적용을 불가능하게 한다.

2.3 스택트레이스 단절 복원 기법

본 연구에서는 Frida 기반 런타임 후킹 기법을 통해 멀티 스레드 환경에서 발생하는 호출 흐름의 단절 문제를 해결하고자 하였다. 구체적으로, `Thread` 객체의 생성자와 시작 메시지를 후킹하여 스레드가 생성되거나 시작될 때의 호출 스택을 캡처하고, 이를 `Thread ID`를 키로 별도 저장소에 보관한다. 이러한 방식으로 스레드 외부 스택과 스레드 내부 실행 시점 스택을 연계하여 호출 흐름을 재구성할 수 있다.

```
at android.telephony.TelephonyManager.getNetworkOperator(  
at com.example.testapp.MainActivity.getNetworkOperatorInT  
at com.example.testapp.MainActivity.handleResultAction$la  
at com.example.testapp.MainActivity.$r8$lambda$6Rj9kcFWPu  
at com.example.testapp.MainActivity$$ExternalSyntheticLam  
at java.lang.Thread.run(Thread.java:1012)  
at java.lang.Thread.start(Native Method)  
at com.example.testapp.MainActivity.handleResultAction(Ma  
at com.example.testapp.MainActivity.calculateResult(MainA  
at com.example.testapp.MainActivity.onCreate$lambda$3(Mai
```

[그림 3] 복원된 스택트레이스 (스레드 직접 생성 시)

[그림 3]의 제안된 기법을 적용해 복원된 스택트레이스를 보여준다. 상단에서는 [그림 2]에서와 동일하게 새로운 스레드에서 타겟 API인 `getNetworkOperator()`까지의 호출 스택을 보여주며, 하단에서는 해당 스레드가 생성될 때까지의 호출 스택을 연결하여 보여준다. 즉, 스레드를 실행한 `com.example.testapp.MainActivity`의 호출 스택을 하단에 표시해, 전체적인 실행 맥락을 이해할 수 있게 한다. `ZygoteInit.main()` 호출은 생략되었으나, 전체 스택트레이스는 `Thread.run()` 호출 전후의 두 부분이 정상적으로 병합되어 멀티 스레드 환경 내 API 호출 흐름을 정확히 재현했음을 확인하였다.

2.4 스레드 풀

스레드 생성과 종료는 운영체제에서 비교적 무거운 작업으로, 빈번한 스레드 생성/소멸은 CPU 및 메모리 자원 낭비를 초래한다. 스레드 풀은 미리 일정 수의 워커 스레드를 생성해 유지하면서 작업 큐에 있는 작업을 효율적으로 분배하여 처리한다. 워커 스레드는 작업 종료 후에도 종료되지 않고 대기 상태로 전환되어 재사용되므로, 스레드 관리 오버헤드를 절감한다. 안드로이드에서는 `Executor`, `Handler` 등 여러 API에서 스레드 풀을 적극 활용하고 있다.

```
at bsj.a(:com.google.android.gms.policy_ads_fdr_dynam  
at brr.run(:com.google.android.gms.policy_ads_fdr_dyna  
at bsk.run(:com.google.android.gms.policy_ads_fdr_dyna  
at java.util.concurrent.ThreadPoolExecutor.runWorker(T  
at java.util.concurrent.ThreadPoolExecutor$Worker.run(  
at java.lang.Thread.run(Thread.java:1012)
```

[그림 4] ThreadPoolExecutor 사용 시 스택트레이스

[그림 4]는 실제 어플리케이션에서 `ContentResolver.query()` 메시지를 후킹한 결과로, 호출 스택 중 `ThreadPoolExecutor` 관련 메서드들이 존재하는 것을 보여준다. 스레드 풀

환경에서는 단순히 Thread ID만으로 외부 호출 흐름과 내부 작업 실행 흐름을 연결하는 것이 불가능하다. 이는 스레드 풀이 동일한 워커 스레드를 반복 재사용하며 여러 Runnable 작업을 처리하기 때문이다. 즉, 하나의 워커 스레드가 서로 다른 여러 작업을 실행할 수 있으므로, Thread ID만으로는 특정 작업과 호출 흐름의 연관 관계를 식별할 수 없다.

이를 극복하기 위해 본 연구에서는 Runnable 객체 자체를 연결 고리로 활용한다. 작업이 스레드 풀에 제출되는 시점에 해당 Runnable 객체를 키로 하여 제출 당시의 스택 트레이스를 저장하고, 실행 시점에는 현재 Thread ID를 통해 실행 중인 Runnable 인스턴스를 찾아낸 뒤, 두 시점의 스택트레이스를 병합한다. 이를 통해 외부에서 작업이 등록된 위치와 내부 스레드에서 실행된 경로를 정확히 연결할 수 있다.

```
at java.util.concurrent.ThreadPoolExecutor.runWorker(T
at java.util.concurrent.ThreadPoolExecutor$Worker.run(
at java.lang.Thread.run(Thread.java:1012)
at java.lang.Thread.start(Native Method)
at java.util.concurrent.ThreadPoolExecutor.addWorker(T
at java.util.concurrent.ThreadPoolExecutor.execute(Thr
at java.util.concurrent.ThreadPoolExecutor.execute(Nat
at com.google.android.gms.ads.internal.util.future.d.e
```

[그림 5] 복원된 스택트레이스 (스레드 풀 사용 시)

[그림 5]는 스레드 풀 환경에서 복원된 스택트레이스 예시이다. 기존에는 Thread.run 지점에서 단절되었던 호출 스택이, 제시한 기법을 통해 작업 제출 시점의 스택트레이스와 연결된 것을 확인할 수 있다.

2.5 그 외 주요 비동기 처리 기법

안드로이드 플랫폼에서는 스레드 풀 이외에도 다양한 비동기 처리 기법들이 사용된다. 대표적인 예로는 다음이 있다.

- AsyncTask: 백그라운드 작업과 UI 갱신을 결합한 추상화 도구로, 내부적으로 스레드 풀과 핸들러를 이용한다.
- 코루틴(Coroutines, Kotlin): 경량 스레드로, 비동기 작업을 직관적으로 표현하며, 기존 스레드 풀과 다른 실행 흐름 구조를 가진다.
- RxJava 등의 리액티브 프로그래밍 라이브

러리: 비동기 데이터 스트림을 처리하며 복잡한 비동기 흐름을 선언적으로 관리한다.

이들 기법은 모두 독특한 비동기 실행 모델과 호출 흐름을 갖고 있어, 향후 본 연구에서는 이러한 다양한 비동기 처리 모델에서 발생하는 호출 흐름 단절을 효과적으로 복원하는 기법 개발을 목표로 한다.

III. 결론

본 연구에서는 멀티 스레드 환경에서 발생하는 스택트레이스 단절 문제를 해결하기 위해, 스레드 생성 시점과 실행 시점의 스택 정보를 연계하는 방법을 제안하였다.

이를 통해 기존 동적 분석에서 추적이 불가능했던 비동기 API 호출의 실제 경로를 복원할 수 있으며, 이는 단순한 악성 앱 분석을 넘어 다음과 같은 분야에 활용 가능하다.

- 동적 행위 기반 앱 프로파일링
- 비정상 API 호출 탐지 및 런타임 디버깅
- 비동기 로직의 성능 분석 및 오류 추적

향후 연구에서는 Coroutine, AsyncTask, RxJava 등 다양한 비동기 처리 프레임워크로 제안 기법을 확장하고, Native 계층(JNI)의 호출 흐름까지 통합하는 것을 목표로 한다.

[참고문헌]

- [1] R. Zhang et al., Hitchhiking Vaccine: Enhancing Botnet Remediation With Remote Code Deployment Reuse, NDSS 2025.
- [2] H. Zhou et al., Uncovering Intent based Leak of Sensitive Data in Android Framework, CCS, 2022.
- [3] Victor Matos and Rebecca Grasser, An Experience on multithreading using android's handler class, CCSC, 2014.